Musterfirma GmbH

Vorlage Pflichtenheft

Bei den nachfolgenden Inhalten handelt es sich lediglich um grobe Vorschläge, an denen Sie sich orientieren können, jedoch nicht müssen. Es empfiehlt sich eine Anpassung an die individuellen Bedürfnisse Ihres Unternehmens.

Projektbezeichnung	Scooter – Konsortium
Projektleiter	Justin Siegfried
Erstellt am	07.09.2026
Letzte Änderung am	14.09.2026
Status	in Bearbeitung
Aktuelle Version	0.2a

Änderungsverlauf

Nr.	Datum	Version	Geänderte Kapitel	Art der Änderung	Autor	Status
1	07.09.2026	0.1a	1 – 5	Erstellung	Linus Popp	i.B.
2	14.09.2026	0.2a	6 - 8	Ergänzung	Justin Siegfried	i.B.
3	21.09.2026	1.0	8	Aktualisierung	Justin Siegfried	i.B
4	28.09.2026	RC	Alle	Korrekturen	Justin Siegfried	Prüfung
5	-	-	-	-	-	-

Inhalt

1 Einleitung 3

2 Allgemeines 3

2.1 Ziel und Zweck des Dokuments 3

2.2 Ausgangssituation 3

2.3 Projektbezug 3

2.4 Abkürzungen 4

2.5 Teams und Schnittstellen 4

3 Konzept 4

3.1 Ziel(e) des Anbieters 4

3.2 Ziel(e) und Nutzen des Anwenders 5

3.3 Zielgruppe(n) 5

4 Funktionale Anforderungen 5

5 Nichtfunktionale Anforderungen 8

5.1 Allgemeine Anforderungen 8

5.2 Gesetzliche Anforderungen 8

5.3 Technische Anforderungen 8

5.4 [weitere nicht funktionale Anforderungen] 9

6 Rahmenbedingungen 9

6.1 Zeitplan 9

6.2 Technische Anforderungen 10

6.3 Problemanalyse 10

6.4 Qualität 11

7 Liefer- und Abnahmebedingungen 11

8 Anhang 12

1 Einleitung

Das vorliegende Pflichtenheft beschreibt die konkrete technische Umsetzung der im Lastenheft definierten Anforderungen an die zu entwickelnde Software. Es legt fest, welche Funktionen umgesetzt werden, welche technischen Komponenten verwendet werden und unter welchen Rahmenbedingungen das Projekt realisiert wird.

Die Anwendung dient der Verwaltung und Simulation von E-Scootern, Firmen und Ladepunkten eines Scooter-Konsortiums. Das System besteht aus einer webbasierten Benutzeroberfläche, einem Backend-Service sowie einer relationalen Datenbank.

2 Allgemeines

2.1 Ziel und Zweck des Dokuments

Dieses Pflichtenheft beschreibt die geplante technische Umsetzung des Projektes „Scooter-Konsortium“. Es konkretisiert die Anforderungen aus dem Lastenheft und dient als Grundlage für die Entwicklung, Prüfung und spätere Abnahme der Software.

Das Dokument beschreibt insbesondere die Systemarchitektur, die funktionalen und nichtfunktionalen Anforderungen, die verwendeten Technologien, den Projektzeitplan sowie die Qualitäts- und Abnahmekriterien.

2.2 Ausgangssituation

Das Projekt soll eine Software zur Verwaltung von E-Scootern innerhalb eines Konsortiums mehrerer Vermieter bereitstellen.

Die Anwendung soll Informationen über Scooter, Firmen und Ladepunkte zentral verwalten. Zusätzlich sollen die aktuellen Positionen der Scooter und Ladepunkte auf einer Karte dargestellt werden.

Die Benutzer sollen über eine übersichtliche Weboberfläche auf die Funktionen des Systems zugreifen können. Das Backend übernimmt die Verarbeitung der Geschäftslogik und die Kommunikation mit der Datenbank.

2.3 Projektbezug

Es handelt sich um ein eigenständiges Ausbildungsprojekt, das von vier Personen der Klasse AD43 durchgeführt wird.

Das Projekt umfasst die Planung, Konzeption, Entwicklung und Dokumentation einer vollständigen Softwarelösung zur Verwaltung und Simulation von E-Scootern.

2.4 Abkürzungen

- i.B. = in Bearbeitung
- API = Application Programming Interface
- ASP.NET = Framework zur Entwicklung von Webanwendungen mit .NET
- SQL = Structured Query Language
- DB = Datenbank
- UML = Unified Modeling Language
- ERD = Entity-Relationship-Diagramm
- UI = User Interface
- CRUD = Create, Read, Update, Delete
- HTTP = Hypertext Transfer Protocol
- REST = Representational State Transfer

2.5 Teams und Schnittstellen

Rolle(n)	Name	Telefon	E-Mail	Team
Projektleiter	Justin Siegfried	+49 176 674206	xy@muserfirma.de	-
Produkt Manager	Marius Groß	+49 183 884402	xy@muserfirma.de	-
Frontend Engineer	Sam Seidel	+49 145 909021	xy@muserfirma.de	-
Backend Engineer	Linus Popp	+49 156 239148	xy@muserfirma.de	-

Die einzelnen Bereiche kommunizieren über definierte Schnittstellen miteinander. Das Frontend kommuniziert mit dem Backend über eine HTTP-basierte API. Das Backend übernimmt die Verarbeitung der Anfragen und den Zugriff auf die SQL-Datenbank.

3 Konzept

3.1 Ziel(e) des Anbieters

Ziel des Anbieters ist die Entwicklung einer zentralen und übersichtlichen Verwaltungssoftware für E-Scooter-Vermieter.

Die Software soll die Verwaltung von Scootern, Firmen und Ladepunkten vereinfachen und eine Grundlage für die spätere Erweiterung des Systems bieten.

Durch die Trennung von Frontend, Backend und Datenbank soll eine strukturierte und wartbare Softwarearchitektur entstehen.

3.2 Ziel(e) und Nutzen des Anwenders

Für die Anwender soll die Verwaltung der E-Scooter möglichst einfach und übersichtlich erfolgen.

Die Anwendung soll unter anderem folgende Vorteile bieten:

- übersichtliche Darstellung aller Scooter,
- Anzeige der aktuellen Scooterpositionen,
- Verwaltung von Firmen,
- Verwaltung von Ladepunkten,
- Anzeige des Ladezustands,
- Simulation von Fahrten,
- Ermittlung geeigneter Ladepunkte,
- Speicherung und Abruf der Daten,
- Auswertung der gespeicherten Daten.

Durch die Kartenansicht sollen Positionen und Zusammenhänge zwischen Scootern und Ladepunkten schnell erkennbar sein.

3.3 Zielgruppe(n)

Die primäre Zielgruppe der Anwendung sind Mitarbeiter eines E-Scooter-Konsortiums, die für die Verwaltung und Überwachung der Scooter verantwortlich sind.

Dazu gehören beispielsweise:

- Mitarbeiter der Scooter-Vermieter,
- Mitarbeiter für die Verwaltung der Ladepunkte,
- Mitarbeiter für die Überwachung der Scooter,
- Administratoren bzw. verantwortliche Mitarbeiter des Konsortiums.

Die Benutzer benötigen keine speziellen Programmierkenntnisse. Die Benutzeroberfläche soll deshalb übersichtlich und verständlich gestaltet werden.

4 Funktionale Anforderungen

Die Anwendung wird als Webanwendung umgesetzt. Die Benutzeroberfläche wird mit Svelte entwickelt. Das Backend wird als ASP.NET-Core-Anwendung in C# umgesetzt. Für die Speicherung der Daten wird eine relationale SQL-Datenbank verwendet.

4.1 Verwaltung von Ladepunkten

- FA01 – Ladepunkt erstellen: Der Benutzer kann einen neuen Ladepunkt anlegen.
- FA02 – Ladepunktdaten: Beim Anlegen werden Name, X-Koordinate, Y-Koordinate und maximale Kapazität erfasst.
- FA03 – Ladepunkt anzeigen: Vorhandene Ladepunkte können in einer Übersicht angezeigt werden.
- FA04 – Ladepunktposition: Die Position eines Ladepunktes wird auf der Karte dargestellt.
- FA05 – Kapazität: Die maximale Anzahl der Ladeplätze sowie die aktuell belegten Ladeplätze werden gespeichert und angezeigt.
- FA06 – Ladepunkt bearbeiten: Die Daten eines bestehenden Ladepunktes können geändert werden.

4.2 Verwaltung von Firmen

- FA07 – Firma erstellen: Der Benutzer kann eine neue Firma anlegen.
- FA08 – Firmendaten: Für eine Firma werden unter anderem Name, Kontaktdaten und Kosten pro gefahrenem Kilometer gespeichert.
- FA09 – Standardkosten: Werden keine individuellen Kilometerkosten angegeben, wird der Standardwert von 0,59 € verwendet.
- FA10 – Ladepunktzuordnung: Einer Firma können Ladepunkte zugeordnet werden.
- FA11 – Firmenübersicht: Alle vorhandenen Firmen können übersichtlich angezeigt werden.
- FA12 – Firma bearbeiten: Bestehende Firmendaten können geändert werden.

4.3 Verwaltung von Scootern

- FA13 – Scooter erstellen: Der Benutzer kann einen neuen Scooter anlegen.
- FA14 – Scooterdaten: Für einen Scooter werden unter anderem Position, Ladezustand, Einnahmen, zurückgelegte Strecke und Fahrtzustand gespeichert.
- FA15 – Initialisierung: Ein neu erstellter Scooter wird standardmäßig mit 100 % Ladezustand und 0 km zurückgelegter Strecke initialisiert.
- FA16 – Firmenzuordnung: Jeder Scooter wird genau einer Firma zugeordnet.
- FA17 – Positionsprüfung: Ein Scooter darf nicht an den gleichen Koordinaten wie ein Ladepunkt angelegt werden.
- FA18 – Scooterübersicht: Alle Scooter können mit ihren relevanten Daten angezeigt werden.
- FA19 – Scooterposition: Die aktuelle Position eines Scooters wird auf der Karte dargestellt.

4.4 Kartenansicht

- FA20 – Kartenintegration: Die Anwendung stellt eine Kartenansicht über eine Maps API bereit.
- FA21 – Scooter anzeigen: Scooter werden entsprechend ihrer gespeicherten Koordinaten auf der Karte dargestellt.
- FA22 – Ladepunkte anzeigen: Ladepunkte werden auf der Karte dargestellt.
- FA23 – Positionsaktualisierung: Änderungen der Position eines Scooters werden in der Kartenansicht aktualisiert.

4.5 Simulation und Fahrten

- FA24 – Scooter auswählen: Der Benutzer kann einen Scooter für eine Simulation auswählen.
- FA25 – Ziel eingeben: Der Benutzer kann ein Ziel anhand von X- und Y-Koordinaten festlegen.

- FA26 – Distanzberechnung: Die Entfernung zwischen aktueller Position und Ziel wird orthogonal berechnet.
- FA27 – Ladezustandsprüfung: Vor einer Fahrt wird überprüft, ob der Ladezustand für die geplante Strecke ausreicht.
- FA28 – Energieverbrauch: Für jede Entfernungseinheit werden zwei Prozentpunkte des Ladezustands verbraucht.
- FA29 – Position aktualisieren: Bei erfolgreicher Fahrt wird die Position des Scooters auf das Ziel gesetzt.
- FA30 – Strecke aktualisieren: Die zurückgelegte Strecke wird um die gefahrene Distanz erhöht.
- FA31 – Niedriger Ladezustand: Bei einem Ladezustand von weniger als 30 % kann der Scooter nicht normal bewegt werden.
- FA32 – Ladepunkt empfehlen: Bei einem Ladezustand von weniger als 30 % wird der nächstgelegene Ladepunkt angezeigt.

4.6 Ladevorgang

- FA33 – Scooter zum Ladepunkt bewegen: Ein Scooter kann zu einem Ladepunkt bewegt werden.
- FA34 – Ladezustand: Beim Beginn des Ladevorgangs wird der Zustand des Scooters auf „lädt“ gesetzt.
- FA35 – Ladeplatzbelegung: Die Anzahl der belegten Ladeplätze wird entsprechend aktualisiert.
- FA36 – Kapazitätsprüfung: Die maximale Kapazität eines Ladepunktes darf nicht überschritten werden.

4.7 Datenbank und Backend

- FA37 – Daten speichern: Die im System erfassten Daten werden dauerhaft in der Datenbank gespeichert.
- FA38 – Daten laden: Das Backend kann gespeicherte Daten aus der Datenbank abrufen.
- FA39 – Daten aktualisieren: Änderungen an Scootern, Firmen und Ladepunkten werden in der Datenbank aktualisiert.
- FA40 – Datenbeziehungen: Beziehungen zwischen Scootern, Firmen und Ladepunkten werden über geeignete Datenbankrelationen abgebildet.
- FA41 – API: Das Frontend kommuniziert über definierte API-Endpunkte mit dem Backend.

4.8 Auswertungen

Das System stellt die im Lastenheft geforderten Auswertungen bereit:

- FA42: Absteigend sortierte Auflistung aller Scooter nach generierten Einnahmen.
- FA43: Anzeige sämtlicher aktuell ladender Scooter.
- FA44: Aggregierte Übersicht der Firmen mit den Einnahmen ihrer Scooter.
- FA45: Anzeige des erfolgreichsten Scooters inklusive der zugehörigen Firmendaten.
- FA46: Anzeige der an einem Ladepunkt geladenen Scooter inklusive Firmennamen.

5 Nichtfunktionale Anforderungen

Nichtfunktionale Anforderungen beschreiben die Qualitätsmerkmale und technischen Eigenschaften des Systems.

5.1 Allgemeine Anforderungen

- NFA01 – Bedienbarkeit: Die Anwendung soll übersichtlich und intuitiv bedienbar sein.
- NFA02 – Verständliche Ausgaben: Fehlermeldungen und Statusinformationen sollen verständlich dargestellt werden.
- NFA03 – Eingabeprüfung: Benutzereingaben sollen auf gültige Werte geprüft werden.
- NFA04 – Datenkonsistenz: Das System darf keine widersprüchlichen Daten erzeugen.
- NFA05 – Wartbarkeit: Der Quellcode soll strukturiert und nachvollziehbar aufgebaut sein.
- NFA06 – Modularität: Frontend, Backend und Datenbankzugriff sollen logisch voneinander getrennt sein.
- NFA07 – Dokumentation: Wichtige Komponenten und Schnittstellen sollen dokumentiert werden.
- NFA08 – Zuverlässigkeit: Fehler sollen kontrolliert behandelt werden.
- NFA09 – Wiederverwendbarkeit: Wiederkehrende Funktionen sollen modular umgesetzt werden.
- NFA10 – Einheitliches Design: Die Benutzeroberfläche soll ein einheitliches und konsistentes Erscheinungsbild besitzen.

5.2 Gesetzliche Anforderungen

Da es sich um eine vereinfachte Schulungs- und Simulationssoftware handelt, werden keine realen Kundendaten oder Zahlungsinformationen benötigt.

- NFA-G01: Es sollen nur für das Projekt erforderliche Daten gespeichert werden.
- NFA-G02: Personenbezogene Daten sollen nur verarbeitet werden, wenn sie für die Anwendung erforderlich sind.
- NFA-G03: Verwendete Software, Bibliotheken und Frameworks müssen entsprechend ihrer jeweiligen Lizenzen verwendet werden.
- NFA-G04: Die Anwendung stellt eine Simulation dar und ersetzt keine reale Vermietungs- oder Abrechnungsplattform.

5.3 Technische Anforderungen

- NFA-T01: Das Backend wird mit C# und ASP.NET Core umgesetzt.
- NFA-T02: Das Frontend wird mit dem Svelte-Framework entwickelt.
- NFA-T03: Für die Benutzeroberfläche werden Tailwind CSS und DaisyUI verwendet.
- NFA-T04: Icons können über Google Icons eingebunden werden.

- NFA-T05: Für die Darstellung der Scooter und Ladepunkte wird die Geoapify eingesetzt.
- NFA-T06: Die Datenhaltung erfolgt in einer relationalen SQL-Datenbank.
- NFA-T07: Die Kommunikation zwischen Frontend und Backend erfolgt über definierte API-Schnittstellen.
- NFA-T08: Das Backend übernimmt die Geschäftslogik und den Zugriff auf die Datenbank.
- NFA-T09: Die Datenbank verwendet geeignete Primär- und Fremdschlüssel zur Sicherstellung der Datenintegrität.
- NFA-T10: Die Anwendung soll modular aufgebaut sein, damit einzelne Komponenten unabhängig voneinander erweitert werden können.

5.4 [weitere nicht funktionale Anforderungen]

- NFA-W01 – Performance: Die Anwendung soll bei der vorgesehenen Anzahl von Scootern, Firmen und Ladepunkten ohne merkliche Verzögerungen arbeiten.
- NFA-W02 – Fehlerbehandlung: Fehlerhafte Eingaben sollen abgefangen und verständlich gemeldet werden.
- NFA-W03 – Sicherheit: Datenbankzugriffe und API-Aufrufe sollen kontrolliert erfolgen.
- NFA-W04 – Skalierbarkeit: Die Anwendung soll grundsätzlich um zusätzliche Scooter, Firmen und Ladepunkte erweitert werden können.
- NFA-W05 – Testbarkeit: Die einzelnen Funktionen sollen anhand definierter Testfälle überprüfbar sein.
- NFA-W06 – Responsive Design: Die Benutzeroberfläche soll sich an unterschiedliche Bildschirmgrößen anpassen.
- NFA-W07 – Einheitlichkeit: Wiederkehrende UI-Elemente sollen einheitlich gestaltet und verwendet werden.
- NFA-W08 – Erweiterbarkeit: Die Architektur soll die spätere Ergänzung weiterer Funktionen ermöglichen.

6 Rahmenbedingungen

Das Projekt wird innerhalb des vorgegebenen Ausbildungszeitraums durchgeführt. Die Entwicklung wird entsprechend der festgelegten Aufgabenverteilung organisiert.

6.1 Zeitplan

Datum	Geplante Tätigkeit
07.09.2026	Projektstart, Aufgabenverteilung und Analyse des Lastenhefts
14.09.2026	Erstellung und Abstimmung des Pflichtenhefts

21.09.2026 Datenbankentwurf, ER-Modell und Softwarearchitektur

28.09.2026 Erstellung der Datenbank und Backend-Grundstruktur

05.10.2026 Entwicklung der Frontend-Grundstruktur

12.10.2026 Implementierung der Backend- und API-Funktionen

19.10.2026 Implementierung der Scooter-Simulation und Kartenansicht

26.10.2026 Implementierung der SQL-Auswertungen und Fehlerbehandlung

02.11.2026 Durchführung der Tests und Fehlerkorrekturen

09.11.2026 Fertigstellung der Dokumentation, Abschlussprüfung und Abgabe

6.2 Technische Anforderungen

Für die Entwicklung und den Betrieb der Software werden folgende Komponenten benötigt:

- Computer bzw. Laptop für die Teammitglieder,
- Entwicklungsumgebung für C# und ASP.NET Core,
- .NET SDK,
- Node.js und npm,
- Svelte bzw. die benötigten Svelte-Werkzeuge,
- SQL-Datenbank,
- Datenbankverwaltungssoftware,
- Versionsverwaltung, beispielsweise Git,
- Software zur Erstellung von UML- und ER-Diagrammen,
- OpenOffice bzw. eine kompatible Textverarbeitung für die Projektdokumentation.

Das Frontend wird als Webanwendung ausgeführt. Das Backend stellt die benötigten API-Schnittstellen zur Verfügung und kommuniziert mit der SQL-Datenbank.

6.3 Problemanalyse

Problem	Möglicher Lösungsansatz
Fehler bei der Verbindung zwischen Frontend und Backend	API-Schnittstellen definieren und mit Testanfragen überprüfen
Fehler bei der Datenbankverbindung	Verbindungskonfiguration und Datenbankzugriff systematisch überprüfen
Ungültige Benutzereingaben	Eingaben validieren und verständliche Fehlermeldungen ausgeben
Inkonsistente Daten	Primär- und Fremdschlüssel sowie geeignete Constraints verwenden
Falsche Zuordnung eines Scooters zu einer Firma	Vor dem Speichern prüfen, ob die ausgewählte Firma existiert
Scooter befindet sich auf einem Ladepunkt	Koordinaten vor dem Speichern vergleichen

Fehler bei der Distanzberechnung	Berechnung anhand definierter Testfälle überprüfen
Falscher Ladezustandsverbrauch	Berechnung mit festgelegten Beispielwerten testen
Überschreitung der Kapazität eines Ladepunktes	Belegung vor Beginn des Ladevorgangs überprüfen
Fehler bei der Kartenanzeige	Koordinatenformat und Maps-API-Anbindung überprüfen

6.4 Qualität

Die Qualität der Software wird durch regelmäßige Tests während der Entwicklung sowie durch eine abschließende Prüfung aller Anforderungen sichergestellt.

Für die Qualitätssicherung werden folgende Maßnahmen durchgeführt:

- Überprüfung der Frontend-Komponenten,
- Überprüfung der Backend-Funktionen,
- Prüfung der API-Schnittstellen,
- Prüfung der Datenbankstruktur,
- Test der Datenbankverbindung,
- Prüfung von gültigen und ungültigen Benutzereingaben,
- Prüfung der Distanzberechnung,
- Prüfung der Ladezustandsberechnung,
- Prüfung des Verhaltens bei einem Ladezustand unter 30 %,
- Prüfung der Ladepunktkapazität,
- Prüfung der Zuordnung von Scootern zu Firmen,
- Prüfung der Zuordnung von Firmen zu Ladepunkten,
- Prüfung der Kartenanzeige,
- Prüfung des Speicherns und Ladens von Daten,
- Durchführung der geforderten SQL-Auswertungen,
- Dokumentation der Testergebnisse.

Ein Test gilt als erfolgreich, wenn das tatsächliche Ergebnis mit dem zuvor festgelegten erwarteten Ergebnis übereinstimmt.

7 Liefer- und Abnahmebedingungen

Das Projekt gilt als abgeschlossen, wenn die im Lastenheft und Pflichtenheft festgelegten wesentlichen Anforderungen umgesetzt, getestet und dokumentiert wurden.

Zum Lieferumfang gehören:

- vollständiger Quellcode des Frontends,
- vollständiger Quellcode des Backends,
- Datenbank einschließlich Datenbankschema,
- SQL-Skripte zur Erstellung und gegebenenfalls Befüllung der Datenbank,

- API-Schnittstellen für die Kommunikation zwischen Frontend und Backend,
- Kartenintegration,
- implementierte Scooter-Simulation,
- SQL-Abfragen für die geforderten Auswertungen,
- Projektdokumentation,
- UML-, ER- und weitere erforderliche Diagramme,
- dokumentierte Testfälle und Testergebnisse,
- gegebenenfalls weitere im Projektverlauf vereinbarte Unterlagen.

Die Abnahme erfolgt durch den Auftraggeber bzw. die zuständige Lehrkraft.

Das Projekt gilt als abgenommen, wenn die wesentlichen Anforderungen nachweislich umgesetzt wurden und die zugehörigen Testfälle erfolgreich durchgeführt werden konnten.

Da es sich um ein schulisches Ausbildungsprojekt handelt, erfolgt keine kommerzielle Preisabrechnung.

8 Anhang

Dem Pflichtenheft können folgende ergänzende Unterlagen beigefügt werden:

- Anhang A: Lastenheft „Scooter-Konsortium“
- Anhang B: Projektplan
- Anhang C: ER-Diagramm
- Anhang D: Relationenmodell / Datenbankschema
- Anhang E: UML-Klassendiagramm
- Anhang F: Use-Case-Diagramm
- Anhang G: Sequenzdiagramme
- Anhang H: Zustandsautomaten
- Anhang I: Programmablaufpläne
- Anhang J: UI-/Frontend-Mockups
- Anhang K: Testfälle und Testergebnisse
- Anhang L: SQL-Skripte
- Anhang M: Beispielhafte Testdaten
- Anhang N: API-Dokumentation
- Anhang O: Architekturübersicht des Systems